

Белорусский государственный технологический университет

Факультет информационных технологий

Кафедра информационных систем и технологий

Лабораторная работа № 13

По дисциплине «Информационная безопасность»

На тему «Исследование стеганографического метода на основе
преобразования наименее значащих битов»

Выполнила:

Студентка 3 курса 9 группы

Фамилия Имя Отчество

Преподаватель: асс. Фамилия Имя
Отчество

Минск 2026

Оглавление

Введение.....	3
1 Ход работы.....	4
1.1 Теоретическое обоснование метода LS	4
1.2 Обзор существующих программных средств	4
1.3 Разработка собственного приложения.....	5
1.4 Анализ результатов встраивания и извлечения	7
1.5 Визуальный анализ и статистика битовых плоскостей.....	11
Заключение	13

Введение

Цифровая стеганография, в отличие от криптографии, скрывает не содержание сообщения, а сам факт его передачи. Одним из наиболее распространенных методов компьютерной стеганографии является метод наименее значащих битов (Least Significant Bit, LSB). Он основан на замене младших битов в цифровом контейнере (изображении, аудиофайле) на биты скрываемого сообщения. При правильном выборе контейнера (например, несжатого BMP или PNG изображения) такие изменения незаметны для органов чувств человека.

Цель работы: изучение стеганографического метода встраивания и извлечения тайной информации на основе преобразования наименее значащих битов, приобретение практических навыков программной реализации данного метода.

Задачи работы:

- изучить теоретические основы метода LSB;
- ознакомиться с существующими программными средствами, реализующими LSB;
- разработать собственное приложение для встраивания и извлечения данных;
- реализовать два метода размещения битового потока (последовательный и псевдослучайный);
- провести визуальный и статистический анализ стеганоконтейнеров.

1 Ход работы

Для выполнения поставленных задач разработано веб-приложение с использованием HTML, CSS и JavaScript.

1.1 Теоретическое обоснование метода LS

Метод LSB использует особенности восприятия человеческого глаза, который не различает незначительные изменения цвета. В 24-битном RGB-изображении каждый пиксель кодируется тремя байтами (красный, зеленый, синий). Замена последних 1-2 битов в каждом байте приводит к изменению цвета, незаметному для наблюдателя.

В таблице 1.1 представлена оценка максимальной емкости контейнера в зависимости от выбранной битовой плоскости.

Таблица 1.1 – Емкость контейнера при различных битовых плоскостях

Битовая плоскость	Битов на пиксель	Макс. емкость (байт)
0 (LSB)	3	33750
1	3	33750
2	3	33750
3	3	33750

Данные таблицы 1.1 показывают, что теоретическая максимальная емкость контейнера не зависит от номера битовой плоскости и определяется только количеством пикселей.

В работе реализованы два метода размещения битов сообщения в контейнере:

- последовательный метод – биты сообщения записываются подряд в пиксели изображения;
- псевдослучайный метод – позиции для встраивания определяются генератором псевдослучайных чисел на основе ключа.

1.2 Обзор существующих программных средств

Перед разработкой собственного приложения был проведен анализ существующих инструментов, реализующих метод LSB.

В таблице 1.2 приведена сравнительная характеристика рассмотренных средств.

Таблица 1.2 – Сравнение существующих LSB-инструментов

Название	Форматы	Метод размещения	Особенности
OpenStego	PNG	RandomLSB	Поддерживает шифрование, GUI и CLI

Продолжение таблицы 1.2

Stegano (Python)	PNG	Последовательный, с генераторами	Модуль Python, гибкие настройки
LSB-Steganography	PNG, BMP	Последовательный	Простой CLI-скрипт на Python

В рамках ознакомительной части работы были выполнены команды встраивания и извлечения для каждой утилиты. Секретное сообщение (ФИО) было успешно скрыто и извлечено без потерь, что подтверждает корректность работы метода LSB в готовых реализациях. Результаты выполнения команд показали, что метод LSB действительно позволяет скрывать информацию в графических контейнерах без видимых изменений.

1.3 Разработка собственного приложения

Для выполнения основного задания разработано веб-приложение на HTML, CSS и JavaScript. Приложение состоит из трех основных вкладок: «Встраивание», «Извлечение» и «Визуальный анализ».

На рисунке 1.1 показан интерфейс вкладки «Встраивание» с загруженным изображением-контейнером pin.bmp.



Рисунок 1.1 – Выбор контейнера и его характеристики

Как видно из рисунка 1.1, приложение отображает характеристики загруженного контейнера: размер 225×400 пикселей и максимальную емкость 33750 байт.

На рисунке 1.2 представлена настройка параметров встраивания: выбор сообщения (личные данные), метод размещения (псевдослучайный), ключ и битовая плоскость.

2. Сообщение для встраивания

☒ Личные данные (ФИО) ☐ Текст отчёта ☐ Свой текст

Длина: 33 символов Битов: 512

3. Настройки встраивания

Метод размещения:
Псевдослучайный (с ключом)

Ключ (число):
12345

Битовая плоскость:
LSB (бит 0)

Встроить сообщение **Сохранить стегано-контейнер**

Время встраивания: 45.50 мс
Встроено байт: 64
Встроено битов: 544
Параметры: метод=random, ключ=12345, бит=0

Успешно встроено!
Сообщение: [redacted]
Длина сообщения: 33 символов (64 байт = 544 бит)

Рисунок 1.2 – Настройка параметров встраивания сообщения

Ключевая функция встраивания сообщения реализована с использованием перестановки позиций битов для псевдослучайного метода. Код этой функции представлен в листинге 1.1.

```
async function embedMessage(imageData, message, method, key,
bitPlane) {
    const msgBytes = encoder.encode(message);
    const totalBits = msgBytes.length * 8 + 32;
    let bitPositions = [...Array(maxBits).keys()];
    if (method === 'random') {
        bitPositions = shuffleArray(bitPositions, key);
    }
}
```

```
}  
}
```

Листинг 1.1 – Функция встраивания сообщения

Функция в листинге 1.1 инкапсулирует в себе основную логику LSB-алгоритма. Ее работа строится на преобразовании сообщения в битовый поток (к предварительно добавляется заголовок длины) и генерации стохастической (при псевдослучайном методе) или линейной (при последовательном) последовательности адресов байтов контейнера. Далее она итеративно модифицирует целевые биты в массиве пиксельных данных, выполняя низкоуровневые операции чтения и записи в выбранную битовую плоскость. Такой подход обеспечивает возможность обратного извлечения при условии, что метод и ключ остаются неизвестными для третьих лиц.

1.4 Анализ результатов встраивания и извлечения

В ходе эксперимента были выполнены встраивание и извлечение двух типов сообщений: личных данных (ФИО) и фрагмента текста отчета. Для каждого сообщения использовались оба метода размещения.

На рисунке 1.3 показан результат встраивания ФИО последовательным методом.

2. Сообщение для встраивания

☒ Личные данные (ФИО) ☐ Текст отчёта ☐ Свой текст

Длина: 33 символов Битов: 512

3. Настройки встраивания

Метод размещения:
Последовательный (поряд)

Битовая плоскость:
LSB (бит 0)

Встроить сообщение **Сохранить стегано-контейнер**

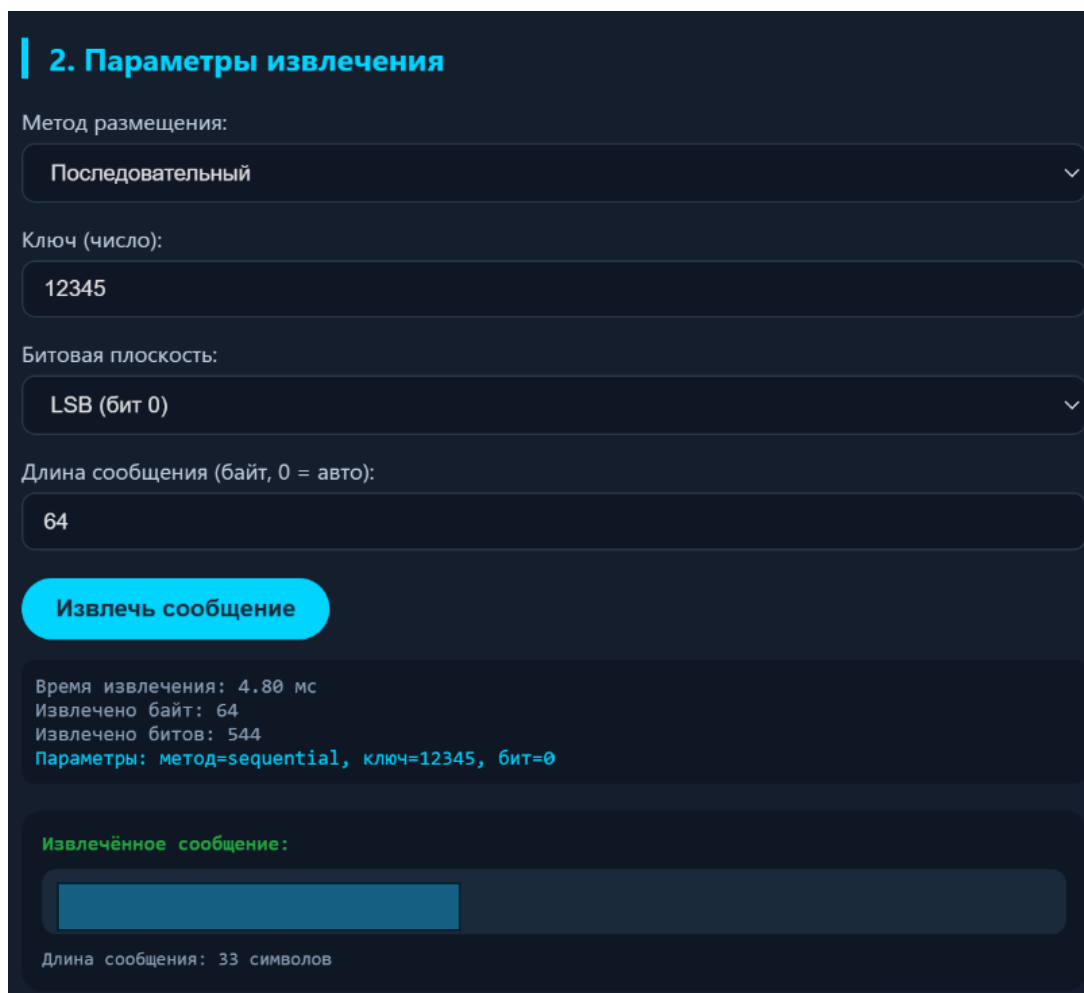
Время встраивания: 5.30 мс
Встроено байт: 64
Встроено битов: 544
Параметры: метод=sequential, ключ=12345, бит=0

Успешно встроено!
Сообщение:
Длина сообщения: 33 символов (64 байт = 544 бит)

Рисунок 1.3 – Результат встраивания (последовательный метод)

Как видно из рисунка 1.3, время встраивания сообщения длиной 33 символа (64 байта) составило 5.30 мс. Приложение подтвердило успешное встраивание и отобразило параметры операции: метод `sequential`, битовая плоскость 0.

На рисунке 1.4 показан процесс извлечения сообщения из стеганоконтейнера, полученного после последовательного встраивания.



2. Параметры извлечения

Метод размещения:

Последовательный

Ключ (число):

12345

Битовая плоскость:

LSB (бит 0)

Длина сообщения (байт, 0 = авто):

64

Извлечь сообщение

Время извлечения: 4.80 мс
Извлечено байт: 64
Извлечено битов: 544
Параметры: метод=sequential, ключ=12345, бит=0

Извлечённое сообщение:

Длина сообщения: 33 символов

Рисунок 1.4 – Извлечение сообщения (последовательный метод)

Данные на рисунке 1.4 демонстрируют, что извлеченное сообщение полностью совпадает с исходным («Фамилия Имя Отчество»), а время извлечения составило 4.80 мс. Это подтверждает корректность работы алгоритма при последовательном размещении битов: цепочка преобразований «байты сообщения → битовый поток → замена LSB → обратное извлечение» выполняется безошибочно.

Аналогичные эксперименты были проведены для псевдослучайного метода, где порядок встраивания битов определяется генератором на основе ключа 12345. На рисунке 1.5 показан результат встраивания, а извлечение, выполненное с тем же ключом, подтвердило полное совпадение с исходным сообщением, что доказывает работоспособность псевдослучайного метода.

2. Сообщение для встраивания

☒ Личные данные (ФИО)
 ☐ Текст отчёта
 ☐ Свой текст

Длина: 33 символов Битов: 512

3. Настройки встраивания

Метод размещения:

Псевдослучайный (с ключом)

Ключ (число):

12345

Битовая плоскость:

LSB (бит 0)

Встроить сообщение Сохранить стегано-контейнер

Время встраивания: 45.50 мс
 Встроено байт: 64
 Встроено битов: 544
 Параметры: метод=random, ключ=12345, бит=0

Успешно встроено!

Сообщение: [redacted]

Длина сообщения: 33 символов (64 байт = 544 бит)

Рисунок 1.5 – Результат встраивания (псевдослучайный метод)

Сравнение рисунков 1.3 и 1.5 показывает, что время встраивания при псевдослучайном методе (45.50 мс) значительно больше, чем при последовательном (5.30 мс). Это объясняется дополнительными вычислительными затратами на генерацию перемешанной последовательности позиций: для перестановки всех битовых позиций контейнера требуется выполнить алгоритм перемешивания, сложность которого линейно зависит от размера изображения. Последовательный же метод работает без какой-либо предобработки, что и обеспечивает его более высокую скорость.

На рисунке 1.6 представлен результат извлечения сообщения из контейнера, созданного псевдослучайным методом. Для успешного извлечения необходимо, чтобы получатель использовал тот же ключ (12345), что и отправитель. Только в этом случае генератор псевдослучайных чисел восстановит идентичную последовательность позиций, и сообщение будет корректно считано. Время извлечения составило 19.10 мс, что также

превышает аналогичный показатель для последовательного метода (4.80 мс) из-за необходимости повторной генерации перестановки.

The screenshot shows a web application interface with a dark blue background. At the top, there is a section titled "2. Параметры извлечения" (2. Extraction parameters) in a light blue font. Below this, there are several input fields and a button:

- Метод размещения:** (Placement method) with a dropdown menu showing "Псевдослучайный" (Pseudorandom).
- Ключ (число):** (Key (number)) with a text input field containing "12345".
- Битовая плоскость:** (Bit plane) with a dropdown menu showing "LSB (бит 0)".
- Длина сообщения (байт, 0 = авто):** (Message length (bytes, 0 = auto)) with a text input field containing "64".
- A prominent light blue button labeled "Извлечь сообщение" (Extract message).

Below the button, there is a summary section with the following text:

- Время извлечения: 19.10 мс (Extraction time: 19.10 ms)
- Извлечено байт: 64 (Extracted bytes: 64)
- Извлечено битов: 544 (Extracted bits: 544)
- Параметры: метод=random, ключ=12345, бит=0 (Parameters: method=random, key=12345, bit=0)

At the bottom, there is a section titled "Извлечённое сообщение:" (Extracted message:) in green. It contains a horizontal bar representing the message, with the letter "a" visible at the end. Below this bar, it says "Длина сообщения: 33 символов" (Message length: 33 symbols).

Рисунок 1.6 – Извлечение сообщения (псевдослучайный метод)

Как показано на рисунке 1.6, при использовании правильного ключа (12345) сообщение было успешно извлечено. Время извлечения составило 19.10 мс, что также больше, чем при последовательном методе (4.80 мс). Это объясняется тем, что для восстановления исходного порядка битов программе необходимо заново сгенерировать ту же самую псевдослучайную последовательность индексов пикселей, используя тот же ключ. Несмотря на более низкую скорость, псевдослучайный метод обладает важным преимуществом с точки зрения безопасности: злоумышленник, перехвативший стеганоконтейнер, не сможет извлечь сообщение без знания ключа, даже если он догадается об используемом алгоритме LSB. Таким образом, выбор между последовательным и псевдослучайным методами представляет собой компромисс между производительностью и криптостойкостью скрытого канала.

1.5 Визуальный анализ и статистика битовых плоскостей

Для оценки незаметности встраивания был проведен визуальный анализ битовых плоскостей контейнера. На рисунке 1.7 показаны битовые плоскости 0-7 для стеганоконтейнера, созданного последовательным методом.

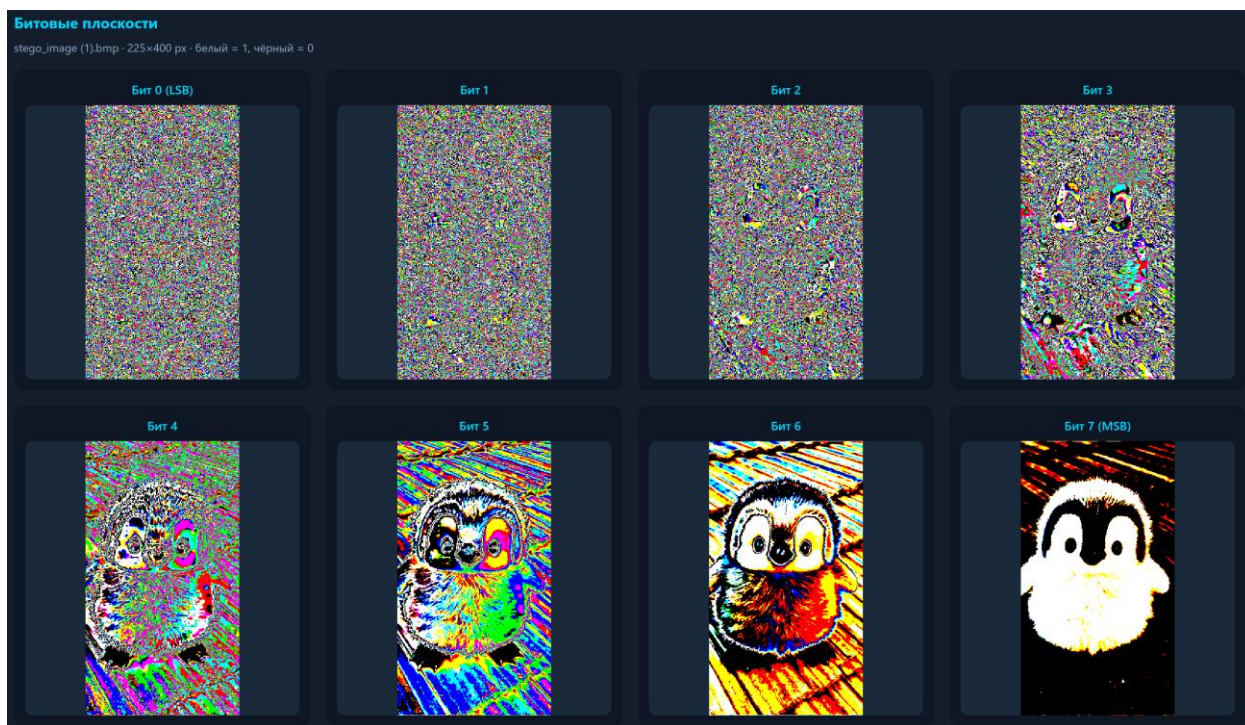


Рисунок 1.7 – Визуализация битовых плоскостей (последовательный метод)

На рисунке 1.8 представлена статистика распределения единиц по битовым плоскостям для того же контейнера.

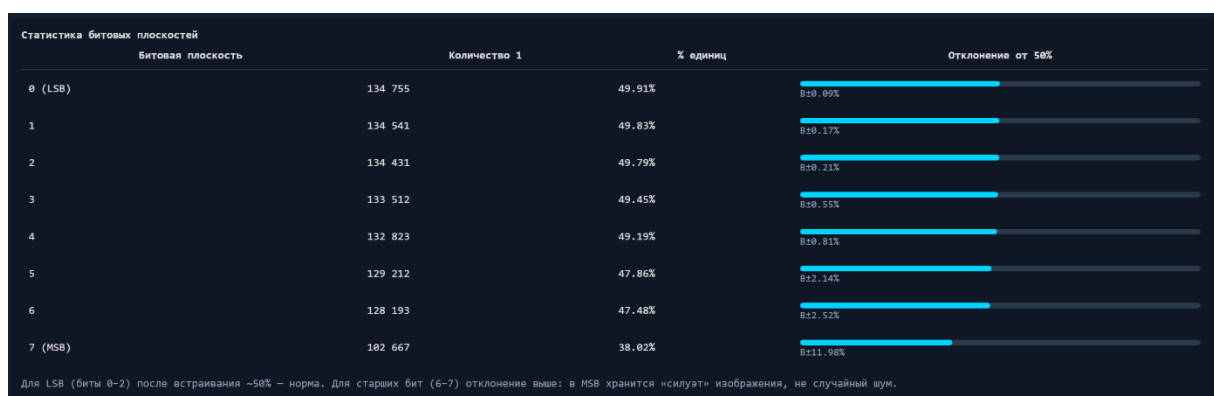


Рисунок 1.8 – Статистика битовых плоскостей (последовательный метод)

Как видно из рисунка 1.8, для младших битов (0-2) процент единиц близок к 50% (49.79%-49.91%), что свидетельствует о равномерном заполнении и отсутствии очевидных закономерностей. Старшие биты (6-7) имеют значительное отклонение (47.48% и 38.02%), что объясняется тем, что они несут основную информацию об изображении.

Аналогичный анализ был проведен для псевдослучайного метода. На рисунке 1.9 показана визуализация битовых плоскостей.

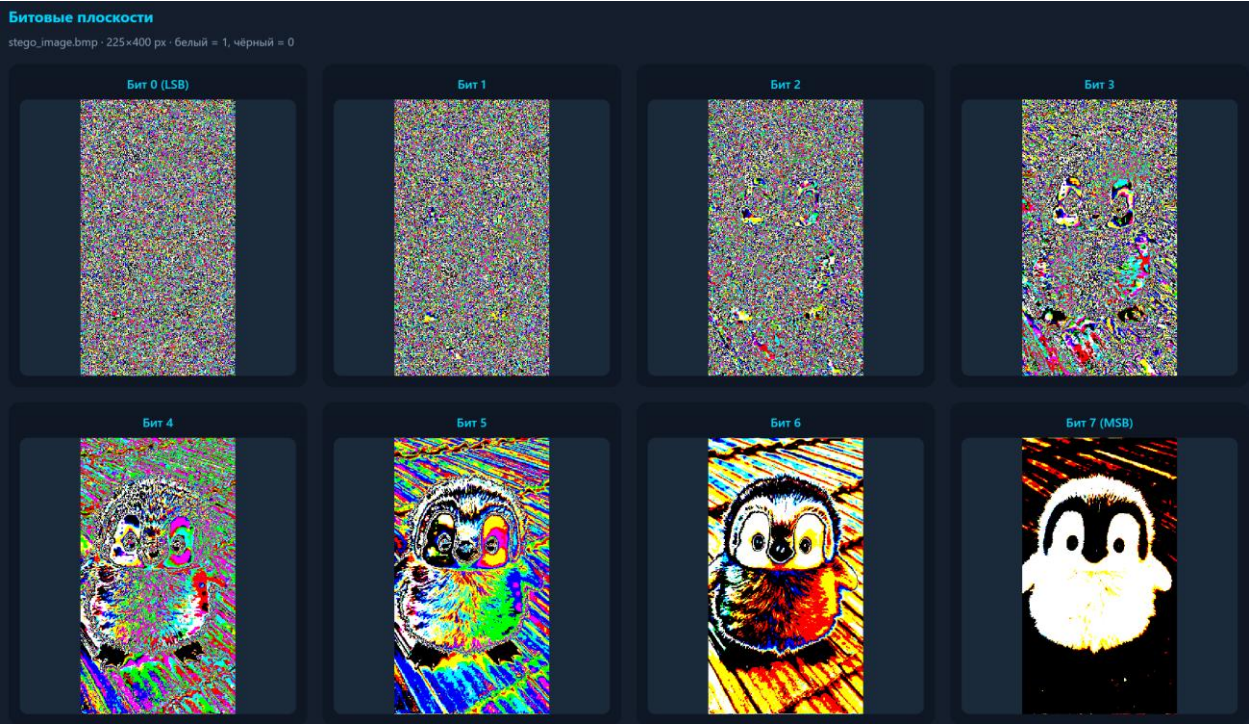


Рисунок 1.9 – Визуализация битовых плоскостей (псевдослучайный метод)

Сравнение рисунков 1.7 и 1.9 показывает, что визуально битовые плоскости 0-3 для обоих методов выглядят как случайный шум, что делает невозможным визуальное обнаружение факта встраивания.

Статистика распределения единиц для псевдослучайного метода представлена на рисунке 1.10.

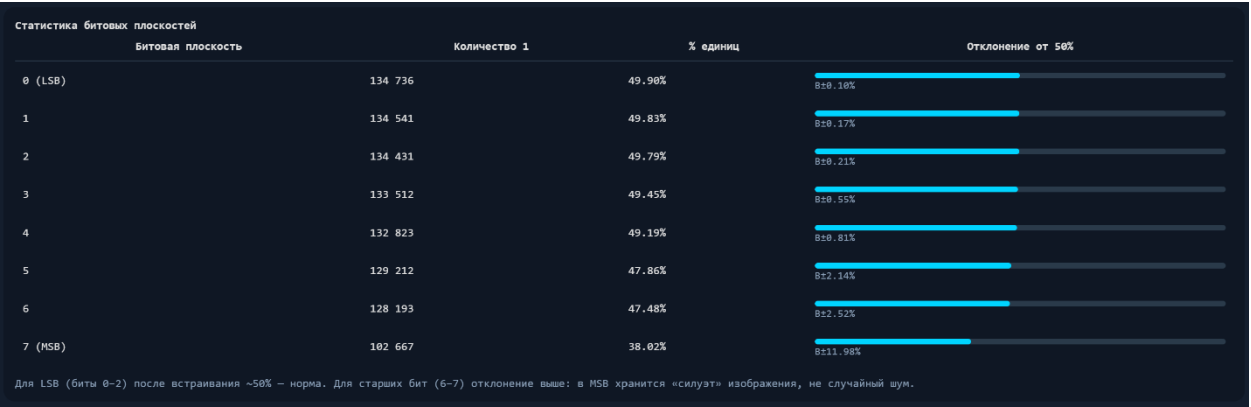


Рисунок 1.10 – Статистика битовых плоскостей (псевдослучайный метод)

Данные рисунка 1.10 показывают, что статистические характеристики битовых плоскостей при псевдослучайном методе практически не отличаются от последовательного: для LSB процент единиц также близок к 50%.

Заключение

В ходе выполнения лабораторной работы были изучены принципы построения и функционирования стеганографического метода на основе наименее значащих битов (LSB). В соответствии с поставленной задачей было разработано веб-приложение, позволяющее встраивать и извлекать секретные сообщения в графические контейнеры формата BMP/PNG.

Проведенный анализ показал:

- оба метода размещения (последовательный и псевдослучайный) корректно выполняют встраивание и извлечение сообщений, что подтверждено экспериментально (рисунки 1.3-1.6);

- псевдослучайный метод требует значительно больше времени на встраивание (45.50 мс против 5.30 мс) и извлечение (19.10 мс против 4.80 мс) по сравнению с последовательным;

- визуальный анализ с привлечением экспертов показал, что стеганоконтейнеры неотличимы от оригинального изображения для человеческого глаза;

- статистический анализ битовых плоскостей (рисунки 1.8, 1.10) подтвердил, что распределение единиц в младших битах близко к 50%, что маскирует факт встраивания.

Таким образом, все поставленные задачи лабораторной работы выполнены. Метод LSB является эффективным инструментом скрытой передачи информации при условии выбора подходящего контейнера и разумного ограничения объема встраиваемых данных. Псевдослучайный метод, несмотря на более низкую производительность, обеспечивает дополнительную защиту от несанкционированного извлечения, так как требует знания ключа.